MODULE 3
Arrays

3.1 Introduction

- ✓ **An array is a collection of similar data elements. These data elements have the same data type. The elements of the array are stored in consecutive memory locations and are referenced by an index (also known as the subscript).**
- ✓ **The subscript is an ordinal number which is used to identify an element of the array.**

3.2 Declaration of Arrays

- ✓ An array must be declared before being used.
- ✓ Arrays are declared using the following **syntax:**

  type name[size];
- ✓ Declaring an array means specifying the following:
  - **Data type**—the kind of values it can store, for example, int, char, float, double, or any other valid data type..
  - **Name**—to identify the array.
  - **Size**—the maximum number of values that the array can hold. i.e., the maximum number of elements that can be stored in the array.

  For example, if we write,

  int marks[10];

then the statement declares marks to be an array containing 10 elements. In C, the array index starts from zero. The first element will be stored in marks[0], second element in marks[1], and so on. Therefore, the last element, that is the 10th element, will be stored in marks[9]. Note that 0, 1, 2, 3 written within square brackets are the subscripts. In the memory, the array will be
stored as shown in Fig. 3.2.

1st element	2nd element	3rd element	4th element	5th element	6th element	7th element	8th element	9th element	10th element
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]	marks[5]	marks[6]	marks[7]	marks[8]	marks[9]

Figure 3.2 Memory representation of an array of 10 elements

- ✓ Figure 3.3 shows how different types of arrays are declared.

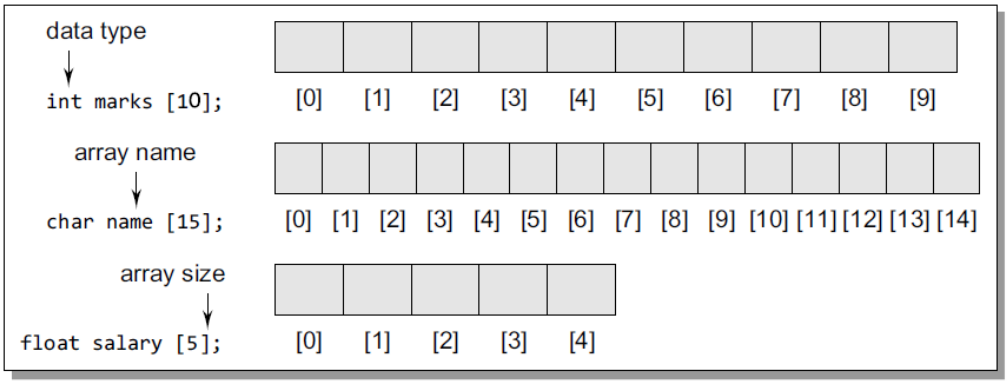


Figure 3.3 Declaring arrays of different data types and sizes

3.3 Accessing the Elements of an Array

- ✓ To access all the elements, **we must use a loop.** That is, **we can access all the elements of an array by varying the value of the subscript into the array.**
- ✓ But note that the subscript must be an integral value or an expression that evaluates to an integral value.
- ✓ As shown in Fig. 3.2, the first element of the array marks[10] can be accessed by writing marks[0]. Now to process all the elements of the array, we use a loop as shown in Fig. 3.4.

```
// Set each element of the array to -1
int i, marks[10];
for(i=0;i<10;i++)
      marks[i] = -1;
```

Figure 3.4 Code to initialize each element of the array to –1

- ✓ Figure 3.5 shows the result of the code shown in Fig. 3.4. The code accesses every individual element of the array and sets its value to –1. In the for loop, first the value of marks[0] is set to –1, then the value of the index (i) is incremented and the next value, that is, marks[1] is set to –1. The procedure continues until all the 10 elements of the array are set to –1.

– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1	– 1
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Figure 3.5 Array `marks` after executing the code given in Fig. 3.4

3.3.1 Calculating the Address of Array Elements

- ✓ The **array name** is a symbolic reference to the **address of the first byte of the array.**
- ✓ When we **use the array name**, we are actually referring to the **first byte of the array.**
- ✓ The **subscript or the index** represents the **offset from the beginning of the array to the element being referenced.**
- ✓ That is, with just the **array name and the index**, **C can calculate the address of any element in the array.**
- ✓ Since an array stores all its data elements in consecutive memory locations, storing just the base address, that is the address of the first element in the array, is sufficient.
- ✓ The address of other data elements can simply be calculated using the base address. The formula to perform this calculation is,

Address of data element, A[k] = BA(A) + w(k – lower_bound)

Here, A is the array,

k is the index of the element of which we have to calculate the address,

BA is the base address of the array A, and

w is the size of one element in memory, for example, size of int is 2.

Example 3.1 Given an array `int marks[]={99,67,78,56,88,90,34,85}`, calculate the address of `marks[4]` if the `base address = 1000`.

Solution

99	67	78	56	**88**	90	34	85
marks[0]	marks[1]	marks[2]	marks[3]	**marks[4]**	marks[5]	marks[6]	marks[7]
1000	1002	1004	1006	**1008**	1010	1012	1014

We know that storing an integer value requires 2 bytes, therefore, its size is 2 bytes.

```
marks[4] = 1000 + 2(4 - 0)
         = 1000 + 2(4) = 1008
```

3.3.2 Calculating the Length of an Array

- ✓ The length of an array is given by the number of elements stored in it.
- ✓ The general formula to calculate the length of an array is

Length = upper_bound – lower_bound + 1

where, upper_bound is the index of the last element and lower_bound is the index of the first element in the array.

Example 3.2 Let `Age[5]` be an array of integers such that

```
Age[0] = 2, Age[1] = 5, Age[2] = 3, Age[3] = 1, Age[4] = 7
```

Show the memory representation of the array and calculate its length.

Solution

The memory representation of the array `Age[5]` is given as below.

2	5	3	1	7
Age[0]	Age[1]	Age[2]	Age[3]	Age[4]

```
Length = upper_bound - lower_bound + 1
```

Here, `lower_bound = 0, upper_bound = 4`

Therefore, `length = 4 - 0 + 1 = 5`

3.4 Storing Values in Arrays

- ✓ When we declare an array, we are just allocating space for its elements; no values are stored in the array.
- ✓ There are three ways to store values in an array. First, to **initialize the array elements during declaration**; second, to **input values for individual elements from the keyboard**; third, to **assign values to individual elements**.
- ✓ This is shown in Fig. 3.6.

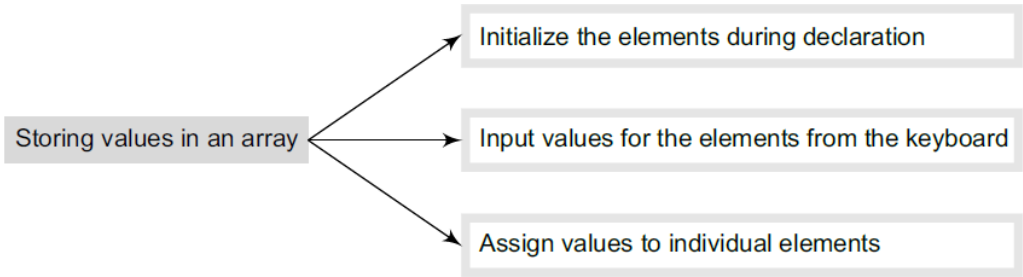


Figure 3.6 Storing values in an array

1. Initializing Arrays during Declaration

- ✓ The elements of an array can be initialized at the **time of declaration**, just as **any other variable.**
- ✓ When an array is initialized, we need **to provide a value for every element in the array.**
- ✓ Arrays are initialized by writing,

type array_name[size]={list of values};

- ✓ Note that the values are written within curly brackets and every value is separated by a comma. It is a compiler error to specify more values than there are elements in the array. When we write,

int marks[5]={90, 82, 78, 95, 88};

- ✓ An array with the name marks is declared that has enough space to store five elements. The first element, that is, marks[0] is assigned value 90. Similarly, the second element of the array, that is marks[1], is assigned 82, and so on. This is shown in Fig. 3.7.

marks[0]	90
marks[1]	82
marks[2]	78
marks[3]	95
marks[4]	88

Figure 3.7 Initialization of array `marks[5]`

- ✓ While initializing the array at the time of declaration, the programmer may omit the size of the array. For example,

int marks[ ]= {98, 97, 90};

The above statement is absolutely legal. Here, the compiler will allocate enough space for all the initialized elements. Note that if the number of values provided is less than the number of elements in the array, the un-assigned elements are filled with zeros. Figure 3.8 shows the initialization of arrays.

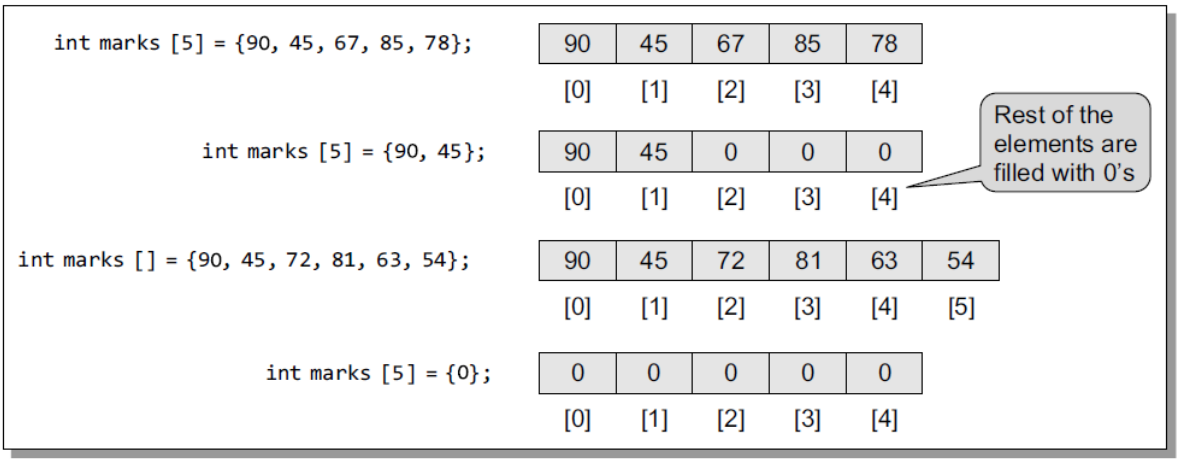


Figure 3.8 Initialization of array elements

2. Inputting Values from the Keyboard

- ✓ An array can be initialized by **inputting values from the keyboard.**
- ✓ In this method, a **while/do–while or a for loop** is executed to input the value for each element of the array. For example, look at the code shown in Fig. 3.9.

```
int i, marks[10];
for(i=0;i<10;i++)
        scanf("%d", &marks[i]);
```

Figure 3.9 Code for inputting each element of the array

- ✓ In the code, we start at the index i at 0 and input the value for the first element of the array. Since the array has 10 elements, we must input values for elements whose index varies from 0 to 9.

3. Assigning Values to Individual Elements

- ✓ The third way is to **assign values to individual elements of the array by using the assignment operator.**
- ✓ Any value that evaluates to the data type as that of the array can be assigned to the individual array element.
- ✓ A simple assignment statement can be written as

  marks[3] = 100;

  Here, 100 is assigned to the fourth element of the array which is specified as marks[3].
- ✓ To copy an array, you must copy the value of every element of the first array into the elements of the second array. Figure 3.10 illustrates the code to copy an array. In Fig. 3.10, the loop accesses each element of the first array and simultaneously assigns its value to the corresponding element of the second array. The index value i is incremented to access the next element in succession. Therefore, when this code is executed, arr2[0] = arr1[0], arr2[1] = arr1[1], arr2[2] = arr1[2], and so on.

```
int i, arr1[10], arr2[10];
arr1[10] = {0,1,2,3,4,5,6,7,8,9};
for(i=0;i<10;i++)
       arr2[i] = arr1[i];
```

Figure 3.10 Code to copy an array at the individual element level

- ✓ For example, if we want to fill an array with even integers (starting from 0), then we will write the code as shown in Fig. 3.11. In the code, we assign to each element a value equal to twice of its index, where the index starts from 0. So after executing this code, we will have arr[0] = 0, arr[1] = 2, arr[2] = 4, and so on.

```
// Fill an array with even numbers
int i,arr[10];
for(i=0;i<10;i++)
       arr[i] = i*2;
```

Figure 3.11 Code for filling an array with even numbers

3.5 Operations on Arrays

- ✓ There are a number of operations that can be performed on arrays.
- ✓ These operations include:
  - **Traversing an array**
  - **Inserting an element in an array**
  - **Searching an element in an array**
  - **Deleting an element from an array**
  - **Merging two arrays**
  - **Sorting an array in ascending or descending order**

3.5.1 Traversing an Array

- ✓ Traversing an array means **accessing each and every element of the array for a specific purpose.**
- ✓ Traversing the data elements of an array A can include **printing every element, counting the total number of elements, or performing any process on these elements**.

✓ The algorithm for array traversal is given in Fig. 3.12.

```
Step 1: [INITIALIZATION] SET I = lower_bound
Step 2: Repeat Steps 3 to 4 while I <= upper_bound
Step 3:      Apply Process to A[I]
Step 4:      SET  I = I + 1
        [END OF LOOP]
Step 5: EXIT
```

Figure 3.12 Algorithm for array traversal

✓ In Step 1, we initialize the index to the lower bound of the array. In Step 2, a while loop is executed. Step 3 processes the individual array element as specified by the array name and index value. Step 4 increments the index value so that the next array element could be processed. The while loop in Step 2 is executed until all the elements in the array are processed, i.e., until I is less than or equal to the upper bound of the array.

Examples:

Write a program to read and display n numbers using an array.

```
#include <stdio.h>
void main()
{
        int i, n, a[10];
        printf("Enter the number of elements in the array : ");
        scanf("%d", &n);
        printf("Enter the array elements:\n");
        for(i=0;i<n;i++)
        {
                scanf("%d",&a[i]);
        }
        printf("The array elements are:\n ");
        for(i=0;i<n;i++)
                printf("%d\n", a[i]);
}
```

Output

```
Enter the number of elements in the array: 5
Enter the array elements:
1
2
3
4
5
The array elements are:
1
2
3
4
5
```

3.5.2 Inserting an Element in an Array

✓ **If an element has to be inserted at the end of an existing array, then we just have to add 1 to the upper_bound and assign the value.** Here, we assume that the memory space allocated for the array

is still available. For example, if an array is declared to contain 10 elements, but currently it has only 8 elements, then obviously there is space to accommodate two more elements. But if it already has 10 elements, then we will not be able to add another element to it.

- ✓ Figure 3.13 shows an algorithm to insert a new element to the end of an array. In Step 1, we increment the value of the upper_bound. In Step 2, the new value is stored at the position pointed by the upper_bound.

```
Step 1: Set upper_bound = upper_bound + 1
Step 2: Set A[upper_bound] = VAL
Step 3: EXIT
```

Figure 3.13 Algorithm to append a new element to an existing array

- ✓ For example, let us assume an array has been declared as

int marks[60];

The array is declared to store the marks of all the students in a class. Now, suppose there are 54 students and a new student comes and is asked to take the same test. The marks of this new student would be stored in marks[55]. Assuming that the student secured 68 marks, we will assign the value as

marks[55] = 68;

- ✓ Consider an array whose elements are arranged in ascending order. Now, if a new element has to be added, it will have to be added probably somewhere in the middle of the array. To do this, we must first find the location where the new element will be inserted and then move all the elements (that have a value greater than that of the new element) one position to the right so that space can be created to store the new value.

Example 3.3 `Data[]` is an array that is declared as `int Data[20];` and contains the following values:

```
Data[] = {12, 23, 34, 45, 56, 67, 78, 89, 90, 100};
```

(a) Calculate the length of the array.
(b) Find the `upper_bound` and `lower_bound`.
(c) Show the memory representation of the array.
(d) If a new data element with the value 75 has to be inserted, find its position.
(e) Insert a new data element 75 and show the memory representation after the insertion.

Solution

(a) Length of the array = number of elements
Therefore, `length of the array = 10`

(b) By default, `lower_bound = 0` and `upper_bound = 9`

(c)

12	23	34	45	56	67	78	89	90	100
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]	Data[8]	Data[9]

(d) Since the elements of the array are stored in ascending order, the new data element will be stored after 67, i.e., at the 6th location. So, all the array elements from the 6th position will be moved one position towards the right to accommodate the new value

(e)

12	23	34	45	56	67	75	78	89	90	100
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]	Data[8]	Data[9]	Data[10]

Algorithm to Insert an Element in the Middle of an Array

- ✓ The algorithm INSERT will be declared as INSERT (A, N, POS, VAL). The arguments are

(a) A, the array in which the element has to be inserted
(b) N, the number of elements in the array
(c) POS, the position at which the element has to be inserted
(d) VAL, the value that has to be inserted

- ✓ In the algorithm given in Fig. 3.14, in Step 1, we first initialize I with the total number of elements in the array. In Step 2, a while loop is executed which will move all the elements having an index greater than POS one position towards right to create space for the new element. In Step 5, we increment the total number of elements in the array by 1 and finally in Step 6, the new value is inserted at the desired position.

```
Step 1: [INITIALIZATION] SET I = N
Step 2: Repeat Steps 3 and 4 while I >= POS
Step 3:          SET A[I + 1] = A[I]
Step 4:          SET I = I - 1
        [END OF LOOP]
Step 5: SET N = N + 1
Step 6: SET A[POS] = VAL
Step 7: EXIT
```

Figure 3.14 Algorithm to insert an element in the middle of an array.

Initial `Data[]` is given as below.

45	23	34	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]

Calling `INSERT(Data, 6, 3, 100)` will lead to the following processing in the array:

45	23	34	12	56	20	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]

45	23	34	12	56	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]

45	23	34	12	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]

45	23	34	100	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]

3.5.3 Deleting an Element from an Array

- ✓ **Deleting an element from an array means removing a data element from an already existing array.**
- ✓ **If the element has to be deleted from the end of the existing array, then we just have to subtract 1 from the upper_bound.**
- ✓ Figure 3.15 shows an algorithm to delete an element from the end of an array.

```
Step 1: SET upper_bound = upper_bound - 1
Step 2: EXIT
```

Figure 3.15 Algorithm to delete the last element of an array

✓ For example, if we have an array that is declared as

int marks[60];

The array is declared to store the marks of all the students in the class. Now, suppose there are 54 students and the student with roll number 54 leaves the course. The score of this student was stored in marks[54]. We just have to decrement the upper_bound. Subtracting 1 from the upper_bound will indicate that there are 53 valid data in the array.

✓ Consider an array whose elements are arranged in ascending order. Now, suppose an element has to be deleted, probably from somewhere in the middle of the array. To do this, we must first find the location from where the element has to be deleted and then move all the elements (having a value greater than that of the element) one position towards left so that the space vacated by the deleted element can be occupied by rest of the elements.

Example 3.4 `Data[]` is an array that is declared as `int Data[10];` and contains the following values:

```
Data[] = {12, 23, 34, 45, 56, 67, 78, 89, 90, 100};
```

(a) If a data element with value 56 has to be deleted, find its position.

(b) Delete the data element 56 and show the memory representation after the deletion.

Solution

(a) Since the elements of the array are stored in ascending order, we will compare the value that has to be deleted with the value of every element in the array. As soon as `VAL = Data[I]`, where `I` is the index or subscript of the array, we will get the position from which the element has to be deleted. For example, if we see this array, here `VAL = 56`. `Data[0] = 12` which is not equal to 56. We will continue to compare and finally get the value of `POS = 4`.

(b)

12	23	34	45	67	78	89	90	100
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]	Data[8]

Algorithm to delete an element from the middle of an array

✓ The algorithm DELETE will be declared as DELETE(A, N, POS). The arguments are:

(a) A, the array from which the element has to be deleted

(b) N, the number of elements in the array

(c) POS, the position from which the element has to be deleted

✓ Figure 3.16 shows the algorithm in which we first initialize I with the position from which the element has to be deleted. In Step 2, a while loop is executed which will move all the elements having an index greater than POS one space towards left to occupy the space vacated by the deleted element. In Step 5, we decrement the total number of elements in the array by 1.

```
Step 1: [INITIALIZATION] SET I = POS
Step 2: Repeat Steps 3 and 4 while I <= N - 1
Step 3:          SET A[I] = A[I + 1]
Step 4:          SET I = I + 1
        [END OF LOOP]
Step 5: SET N = N - 1
Step 6: EXIT
```

Figure 3.16 Algorithm to delete an element from the middle of an array

✓ Calling DELETE (Data, 6, 2) will lead to the following processing in the array.

45	23	34	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]

45	23	12	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]

45	23	12	56	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]

45	23	12	56	20	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]

45	23	12	56	20
Data[0]	Data[1]	Data[2]	Data[3]	Data[4]

Figure 3.17 Deleting elements from an array

3.5.4 Merging Two Arrays

✓ **Merging two arrays in a third array means first copying the contents of the first array into the third array and then copying the contents of the second array into the third array. Hence, the merged array contains the contents of the first array followed by the contents of the second array.** This operation is shown in Fig 3.18.

Array 1-	90	56	89	77	69							
Array 2-	45	88	76	99	12	58	81					
Array 3-	90	56	89	77	69	45	88	76	99	12	58	81

Figure 3.18 Merging of two unsorted arrays

✓ If we have two sorted arrays and the resultant merged array also needs to be a sorted one, then the task of merging the arrays becomes a little difficult. The task of merging can be explained using Fig. 3.19.

Array 1-	20	30	40	50	60							
Array 2-	15	22	31	45	56	62	78					
Array 3-	15	20	22	30	31	40	45	50	56	60	62	78

Figure 3.19 Merging of two sorted arrays

3.5.5 Searching for a Value in an Array

✓ **Searching means to find whether a particular value is present in an array or not.**

✓ If the **value is present** in the array, then **searching is said to be successful** and the searching process gives the **location of that value in the array**. However, if the **value is not present** in the array, the

searching process displays an **appropriate message** and in this case **searching is said to be unsuccessful**.

- ✓ There are two popular methods for searching the array elements: **linear search and binary search.**
- ✓ If the elements of the array are arranged in ascending order, then binary search should be used, as it is more efficient for sorted lists in terms of complexity.

1. Linear Search

- ✓ Linear search, also called as **sequential search**, is a very simple method used for searching an array for a particular value.
- ✓ **It works by comparing the value to be searched with every element of the array one by one in a sequence until a match is found.**
- ✓ Linear search is mostly used to search an **unordered list of elements** (array in which data elements are not sorted).
- ✓ For example, if an array A[] is declared and initialized as, int A[ ] = {10, 8, 2, 7, 3, 4, 9, 1, 6, 5}; and the value to be searched is VAL = 7, then searching means to find whether the value '7' is present in the array or not. If yes, then it returns the position of its occurrence. Here, POS = 3 (index starting from 0).
- ✓ Figure 14.1 shows the algorithm for linear search. In Steps 1 and 2 of the algorithm, we initialize the value of POS and I. In Step 3, a while loop is executed that would be executed till I is less than N (total number of elements in the array). In Step 4, a check is made to see if a match is found between the current array element and VAL. If a match is found, then the position of the array element is printed, else the value of I is incremented to match the next element with VAL. However, if all the array elements have been compared with VAL and no match is found, then it means that VAL is not present in the array.

```
LINEAR_SEARCH(A, N, VAL)
Step 1: [INITIALIZE] SET POS = -1
Step 2: [INITIALIZE] SET I = 1
Step 3:     Repeat Step 4 while I<=N
Step 4:           IF A[I] = VAL
                        SET POS = I
                        PRINT POS
                        Go to Step 6
                  [END OF IF]
                   SET I = I + 1
            [END OF LOOP]
Step 5: IF POS = -1
        PRINT "VALUE IS NOT PRESENT
        IN THE ARRAY"
        [END OF IF]
Step 6: EXIT
```

Figure 14.1 Algorithm for linear search

Example:

Write a program to search an element in an array using the linear search technique.

```
#include <stdio.h>
```

```
void main()
{
        int array[100], key, i, n;

        printf("Enter number of elements in array:\n");
        scanf("%d", &n);

        printf("Enter elements of array:\n");
        for (i = 0; i < n; i++)
                scanf("%d", &array [ i ] );

        printf("Enter a number to search\n");
        scanf("%d", &key);

        for (i = 0; i < n; i++)
        {
                if (array[i] == key)
                {
                        printf("%d is present at location %d.\n", key, i+1);
                        break;
                }
        }
        if (i == n)
                printf("%d isn't present in the array.\n", key);
}
```

```
Output:
Enter number of elements in
array:
5
Enter elements of array:
100
25
35
30
56
Enter a number to search:
30
30 is present at location 4
```

2. Binary Search

- ✓ Binary search is a **fast searching algorithm**.
- ✓ This search algorithm works on the principle of **divide and conquers**.
- ✓ Binary search is a searching algorithm that works efficiently with a **sorted list**.
- ✓ **Binary search begins by comparing the middle element of the array with the key value. If the key value matches the middle element, its position in the array is returned. If the key value is less than the middle element, the search continues in the lower half of the array. If the key value is greater than the middle element, the search continues in the upper half of the array.**
- ✓ Now, let us consider how this mechanism is applied to search for a value in a sorted array.
- ✓ In this algorithm, we see that BEG and END are the beginning and ending positions of the segment that we are looking to search for the element. MID is calculated as (LOW + HIGH)/2. Initially, LOW = lower_bound and HIGH = upper_bound. The algorithm will terminate when KE = A[MID]. When the algorithm ends, we will set POS = MID. POS is the position at which the value is present in the array. However, if KEY is not equal to A[MID], then the values of LOW, HIGH, and MID will be changed depending on whether VAL is smaller or greater than A[MID].

(a) If KEY < A[MID], then KEY will be present in the left segment of the array. So, the value of HIGH will be changed as HIGH = MID – 1.

(b) If KEY > A[MID], then KEY will be present in the right segment of the array. So, the value of BEG will be changed as LOW = MID + 1.

(c) Finally, if KEY is not present in the array, then eventually, HIGH will be less than LOW. When this happens, the algorithm will terminate and the search will be unsuccessful. Figure 14.2 shows the algorithm for binary search.

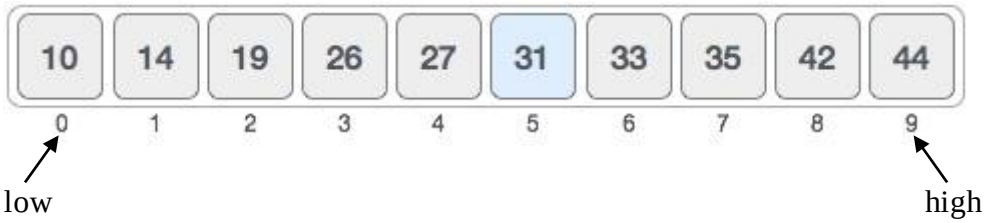
BINARY_SEARCH(A, lower_bound, upper_bound, KEY)

Step 1: [INITIALIZE] SET LOW = lower_bound
HIGH = upper_bound, POS = - 1
Step 2: Repeat Steps 3 and 4 while LOW <= HIGH
Step 3: SET MID = (LOW + HIGH)/2
Step 4: IF KEY = A[MID]
SET POS = MID
PRINT POS
Go to Step 6
ELSE IF KEY > A[MID]
SET LOW = MID + 1
ELSE
SET HIGH = MID - 1
[END OF IF]
[END OF LOOP]
Step 5: IF POS = -1
PRINT "VALUE IS NOT PRESENT IN THE ARRAY"
[END OF IF]
Step 6: EXIT

How Binary Search Works?

- ✓ For a binary search to work, it is mandatory for the target array to be sorted. The following is our sorted array and let us assume that we need to search the location of key value **31** using binary search.

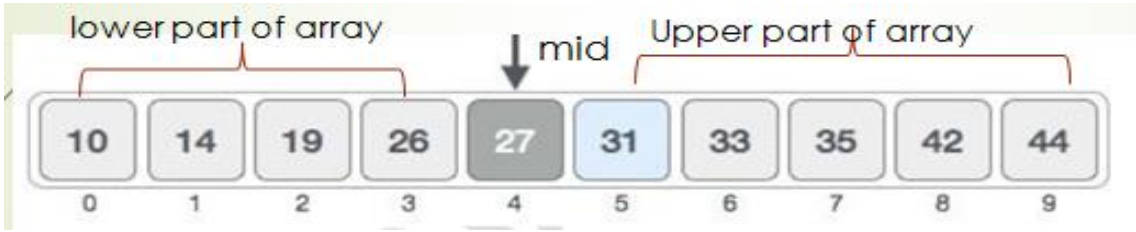
Here n=10, key=31



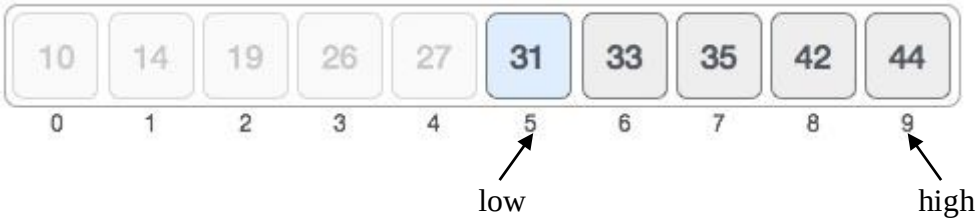
First, we shall determine middle position of the array by using this formula −

mid = (low + high) / 2

Here it is, mid=(0 + 9) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.



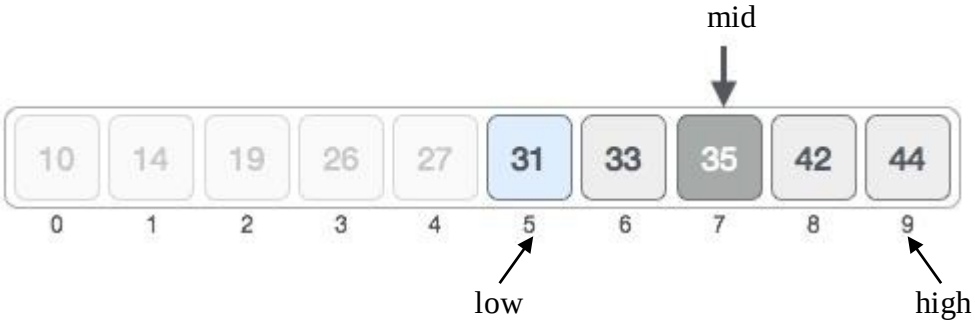
- ✓ Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. Since the key element is greater than the middle element, we should search the key element in the upper part of the array



We change our low to mid + 1 and find the new mid value again.

low = mid + 1=4+1=5
mid = (low + high) /2= (5+9)/2 =7

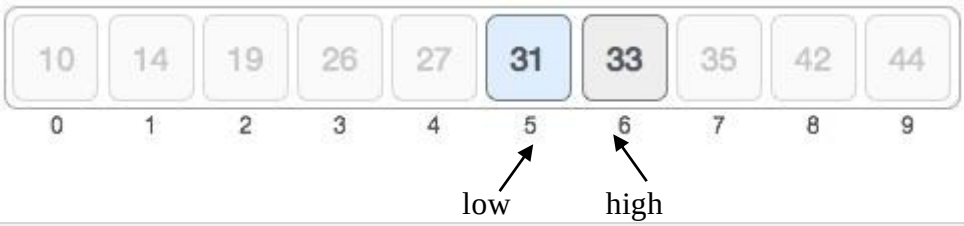
- ✓ Our new mid is 7 now. We compare the value stored at location 7 with our key value 31.



- ✓ The value stored at location 7 is not a match; rather it is more than what we are looking for. Since the key element is less than the middle element, we should search the key element in the lower part of the array.

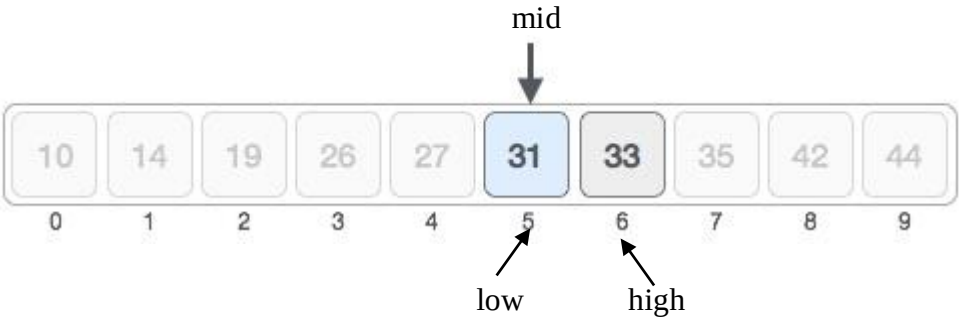
We change our high to mid - 1 and find the new mid value again.

high=mid-1 = mid -1=7-1=6



mid = (low + high) / 2 = (5+6)/2 =5

- ✓ Hence, we calculate the mid again. This time it is 5.



- ✓ We compare the value stored at location 5 with our key value. We find that it is a match.

- ✓ We conclude that the key value 31 is stored at position mid+1= 5+1=6.

- ✓ Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Example: C Program to search key elements in array using binary search algorithms.

```
#include <stdio.h>
#include<stdlib.h>

void main()
{
        int  i, low, high, mid, n, key, a[100];
        printf("Enter number of elements in array:\n");
        scanf("%d",&n);
        printf("Enter integer numbers in ascending order:\n");
        for (i = 0; i< n; i++)
        scanf("%d",&a[i]);
        printf("Enter value to Search\n");
        scanf("%d", &key);
        low = 0;
        high = n - 1;
        while (low <= high)
        {
                mid = (low+high)/2;
                if (key == a[mid])
                {
                  printf("%d found at location %d.\n", key, mid+1);
                  exit(0);
                }
                if (key > a[mid] )
                        low = mid + 1;
                if (key < a[mid])
                        high = mid - 1;
        }
        printf(" %d is Not found! \n", key);

}
```

```
Output:
Enter number of elements
in array:
5
Enter integer numbers in
ascending order:
10
25
35
50
65
Enter a number to search:
65
65 is present at location 5
```

3.6 Passing Arrays to Functions

- ✓ Like variables of other data types, we can also pass an array to a function.
- ✓ In some situations, you may want to pass **individual elements of the array**; while in other situations, you may want to pass the **entire array** as shown in Fig. 3.20.

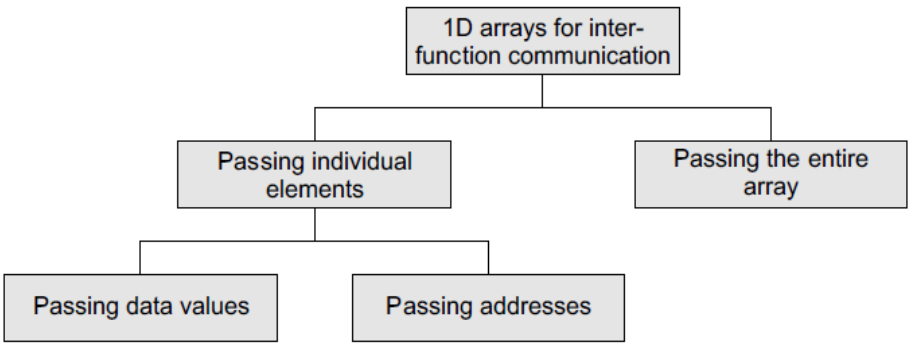


Figure 3.20 One dimensional arrays for inter-function communication

3.6.1 Passing Individual Elements

- ✓ The individual elements of an array can be passed to a function by passing either their **data values or addresses.**

1. Passing Data Values

- ✓ Individual elements can be passed in the same manner as we **pass variables of any other data type.**
- ✓ The condition is just that the **data type of the array element must match with the type of the function parameter.**
- ✓ Look at Fig. 3.21(a) which shows the code to pass an individual array element by passing the data value.

Calling function	Called function

`main()`
`{`
`int arr[5] ={1, 2, 3, 4, 5};`
`func(arr[3]);`
`}`
	`void func(int num)`
	`{`
	`printf("%d", num);`
	`}`

Figure 3.21(a) Passing values of individual array elements to a function

- ✓ In the above example, only one element of the array is passed to the called function. This is done by using the index expression. Here, arr[3] evaluates to a single integer value.

2. Passing Addresses

- ✓ Like ordinary variables, we can pass the **address of an individual array element by preceding the indexed array element with the address operator**.
- ✓ Therefore, to pass the address of the fourth element of the array to the called function, we will write &arr[3].
- ✓ However, in the called function, the value of the array element must be accessed using the indirection (*) operator. Look at the code shown in Fig. 3.21(b).

Calling function	Called function

`main()`
`{`
`int arr[5] ={1, 2, 3, 4, 5};`
`func(&arr[3]);`
`}`
	`void func(int *num)`
	`{`
	`printf("%d", *num);`
	`}`

Figure 3.21(b) Passing addresses of individual array elements to a function

3.6.2 Passing the Entire Array

- ✓ In C the array name refers to the first byte of the array in the memory.
- ✓ **The address of the remaining elements in the array can be calculated using the array name and the index value of the element.**
- ✓ Therefore, when we need to pass an entire array to a function, we can simply pass the name of the array.
- ✓ Figure 3.22 illustrates the code which passes the entire array to the called function.

Calling function	Called function

`main()`
`{`
`int arr[5] ={1, 2, 3, 4, 5};`
`func(arr);`
`}`
	`void func(int arr[5])`
	`{`
	`int i;`
	`for(i=0;i<5;i++)`
	`printf("%d", arr[i]);`
	`}`

Figure 3.22 Passing entire array to a function

3.7 Two-Dimensional Arrays

- ✓ **A two-dimensional array is specified using two subscripts where the first subscript denotes the row and the second denotes the column.**
- ✓ The C compiler treats a two-dimensional array as an array of one-dimensional arrays.
- ✓ Figure 3.26 shows a two-dimensional array which can be viewed as an array of arrays.

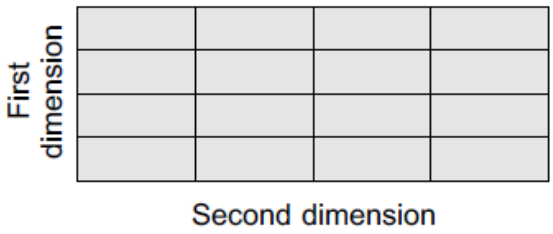


Figure 3.26 Two-dimensional array

3.7.1 Declaring Two-dimensional Arrays

- ✓ Any array must be declared before being used. The declaration statement tells the compiler the **name of the array, the data type of each element in the array, and the size of each dimension.**
- ✓ A two-dimensional array is declared as:

  data_type array_name[row_size][column_size];
- ✓ For example, if we want to store the marks obtained by three students in five different subjects, we can declare a two dimensional array as:

  int marks[3][5];
- ✓ In the above statement, a two-dimensional array called marks has been declared that has m(3) rows and n(5) columns. The first element of the array is denoted by marks[0][0], the second element as marks[0][1], and so on. Here, marks[0][0] stores the marks obtained by the first student in the first subject, marks[1][0] stores the marks obtained by the second student in the first subject.
- ✓ The pictorial form of a two-dimensional array is shown in Fig. 3.27.

Rows \ Columns	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	marks[0][0]	marks[0][1]	marks[0][2]	marks[0][3]	marks[0][4]
Row 1	marks[1][0]	marks[1][1]	marks[1][2]	marks[1][3]	marks[1][4]
Row 2	marks[2][0]	marks[2][1]	marks[2][2]	marks[2][3]	marks[2][4]

Figure 3.27 Two-dimensional array

- ✓ Hence, we see that a 2D array is treated as a collection of 1D arrays. Each row of a 2D array corresponds to a 1D array consisting of n elements, where n is the number of columns. To understand this, we can also see the representation of a two-dimensional array as shown in Fig. 3.28.

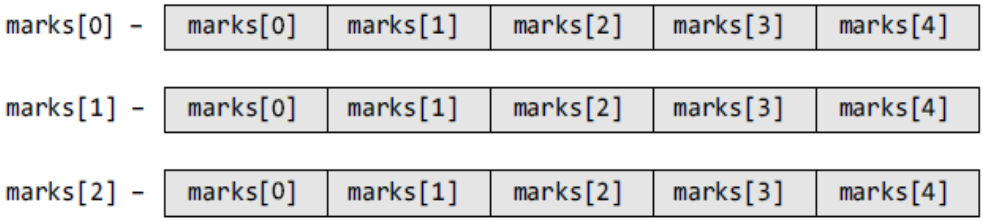


Figure 3.28 Representation of two-dimensional array `marks[3][5]`

- ✓ There are two ways of storing a two-dimensional array in the memory. The first way is the **row major order** and the second is the **column major order**.
- ✓ In a **row major order**, **the elements of the first row are stored before the elements of the second and third rows.** That is, the elements of the array are stored row by row where n elements of the first row will occupy the first n locations. This is illustrated in Fig. 3.29.

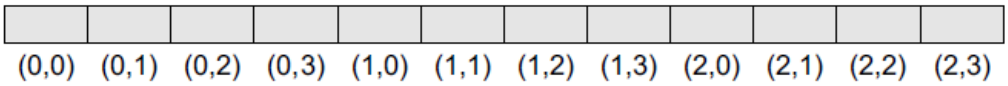


Figure 3.29 Elements of a 3 × 4 2D array in row major order

- ✓ However, when we store the elements in a **column major order, the elements of the first column are stored before the elements of the second and third column.** That is, the elements of the array are stored column by column where m elements of the first column will occupy the first m locations. This is illustrated in Fig. 3.30.

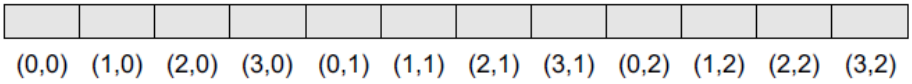


Figure 3.30 Elements of a 4 × 3 2D array in column major order

- ✓ If the array elements are stored in column major order,

  Address(A[I][J]) = Base_Address + w{M (J – 1) + (I – 1)}

  And if the array elements are stored in row major order,

  Address(A[I][J]) = Base_Address + w{N (I – 1) + (J – 1)}

where w is the number of bytes required to store one element, N is the number of columns, M is the number of rows, and I and J are the subscripts of the array element.

Example 3.5 Consider a 20 × 5 two-dimensional array `marks` which has its base address = 1000 and the size of an element = 2. Now compute the address of the element, `marks[18][4]` assuming that the elements are stored in row major order.

Solution

```
Address(A[I][J]) = Base_Address + w{N (I - 1) + (J - 1)}
Address(marks[18][4]) = 1000 + 2 {5(18 - 1) + (4 - 1)}
                      = 1000 + 2 {5(17) + 3}
                      = 1000 + 2 (88)
                      = 1000 + 176 = 1176
```

3.7.2 Initializing Two-Dimensional Arrays

- ✓ A two-dimensional array is initialized in the **same way as a one-dimensional array is initialized**. For example,

  int marks[2][3]={90, 87, 78, 68, 62, 71};

- ✓ Note that the initialization of a two-dimensional array is done row by row. The above statement can also be written as:

  int marks[2][3]={{90,87,78},{68, 62, 71}};

- ✓ The above two-dimensional array has two rows and three columns. First, the elements in the first row are initialized and then the elements of the second row are initialized. Therefore,

  marks[0][0] = 90 marks[0][1] = 87 marks[0][2] = 78

  marks[1][0] = 68 marks[1][1] = 62 marks[1][2] = 71

- ✓ In case of one-dimensional arrays, we have discussed that if the array is completely initialized, we may omit the size of the array. The same concept can be applied to a two-dimensional array, except that only the **size of the first dimension can be omitted**. Therefore, the declaration statement given below is valid.

```
int marks[][3]={{90,87,78},{68, 62, 71}};
```

- ✓ In order to initialize the **entire two-dimensional array to zeros**, simply specify the first value as zero. That is,

```
int marks[2][3] = {0};
```

- ✓ The **individual elements** of a two-dimensional array can be initialized using the **assignment operator** as shown here.

```
marks[1][2] = 79;
or
marks[1][2] = marks[1][1] + 10;
```

3.7.3 Accessing the Elements of Two-dimensional Arrays

- ✓ **The elements of a 2D array are stored in contiguous memory locations.**
- ✓ Since the two-dimensional array contains two subscripts, we will use **two for loops to scan the elements**.
- ✓ The first for loop will scan each row in the 2D array and the second for loop will scan individual columns for every row in the array.

Example:
Write a C program to read and print the elements of a 2D array.

```
#include <stdio.h>

void main()
{
	int arr[2][2], i, j, m, n;
	printf("Enter the size of the array:");
	scanf("%d%d",&m,&n);
	printf("Enter the elements of the array:\n);
	for(i=0;i<m;i++)
	{
		for(j=0;j<n;j++)
		{
			scanf("%d", &arr[i][j]);
		}
	}
	printf("The elements of the array are:\n);
	for(i=0;i<m;i++)
	{
		for(j=0;j<n;j++)
		{
			printf("%d\t", arr[i][j]);
		}
		printf("\n");
	}
}
```

3.8 Operations on Two-Dimensional Arrays

- ✓ Two-dimensional arrays can be used to **implement the mathematical concept of matrices**.
- ✓ In mathematics, a **matrix is a grid of numbers, arranged in rows and columns.**
- ✓ Thus, using two dimensional arrays, we can perform the following operations on an m×n matrix:

1. Transpose

- ✓ Transpose of an m × n matrix A is given as a n × m matrix B, where

$$B_{i,j} = A_{j,i}.$$

2. Sum

- ✓ Two matrices that are compatible with each other can be added together, storing the result in the third matrix. Two matrices are said to be compatible when they have the same number of rows and columns. The elements of two matrices can be added by writing:

$$C_{i,j} = A_{i,j} + B_{i,j}$$

3. Difference

- ✓ Two matrices that are compatible with each other can be subtracted, storing the result in the third matrix. Two matrices are said to be compatible when they have the same number of rows and columns. The elements of two matrices can be subtracted by writing:

$$C_{i,j} = A_{i,j} - B_{i,j}$$

4. Product

- ✓ Two matrices can be multiplied with each other if the number of columns in the first matrix is equal to the number of rows in the second matrix. Therefore, m × n matrix A can be multiplied with a p × q matrix B if n=p. The dimension of the product matrix is m × q. The elements of two matrices can be multiplied by writing:

$$C_{i,j} = \Sigma\, A_{i,k}\, B_{k,j} \text{ for } k = 1 \text{ to } n$$

Example:

1. Write a C program to transpose a 3 × 3 matrix.

```
#include <stdio.h>

void main()
{
	int i, j, mat[3][3], transposed_mat[3][3];
	printf("\n Enter the elements of the matrix ");
	for(i=0;i<3;i++)
	{
		for(j=0;j<3;j++)
		{
			scanf("%d", &mat[i][j]);
		}
	}
	printf("\n The elements of the matrix are ");
	for(i=0;i<3;i++)
	{
		for(j=0;j<3;j++)
		{
```

```
                printf("%d\t", mat[i][j]);
            }
            printf("\n");
        }
        for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
                transposed_mat[i][j] = mat[j][i];
        }


        printf("\n The elements of the transposed matrix are ");
        for(i=0;i<3;i++)
        {
            for(j=0;j<3;j++)
            {
                printf("%d\t",transposed_ mat[i][j]);
            }
            printf("\n");
        }
}
```

Output

Enter the elements of the matrix
1 2 3 4 5 6 7 8 9
The elements of the matrix are
1 2 3
4 5 6
7 8 9
The elements of the transposed matrix are
1 4 7
2 5 8
3 6 9

2. Write a program to input two m × n matrices and then calculate the sum of their corresponding elements and store it in a third m × n matrix.

```
#include <stdio.h>
#include<stdlib.h>

void main()
{
        int i, j, m, n,  p, q, a[5][5], b[5][5], c[5][5];
        printf("\n Enter the number of rows and columns in the first matrix : ");
        scanf("%d%d",&m,&n);

        printf("\n Enter the number of rows and columns in the second matrix : ");
        scanf("%d%d",&p,&q);

        if(m != p || n != q)
        {
            printf("\n Number of rows and columns of both matrices must be equal");
            exit(0);
```

```
        }

        printf("\n Enter the elements of the first matrix ");
        for(i=0;i<m;i++)
        {
                for(j=0;j<n;j++)
                {
                        scanf("%d",&a[i][j]);
                }
        }

        printf("\n Enter the elements of the second matrix ");
        for(i=0;i<p;i++)
        {
                for(j=0;j<q;j++)
                {
                        scanf("%d",&b[i][j]);
                }
        }

        for(i=0;i<m;i++)
        {
                for(j=0;j<n;j++)
                        c[i][j] = a[i][j] + b[i][j];
        }

        printf("\n The elements of the resultant matrix are ");
        for(i=0;i<m;i++)
        {
                for(j=0;j<n;j++)
                {
                        printf("%d\t ", c[i][j]);
                }
                printf("\n");
        }
}
```

3.9 Passing two-dimensional arrays to functions

- ✓ There are three ways of passing a two-dimensional array to a function.
- ✓ First, we can **pass individual elements of the array**. This is exactly the same as passing an element of a one-dimensional array.
- ✓ Second, we can pass a **single row of the two-dimensional array.** This is equivalent to passing the entire one-dimensional array to a function.
- ✓ Third, we can pass the **entire two-dimensional array to the function.**
- ✓ Figure 3.31 shows the three ways of using two-dimensional arrays for inter-function communication.

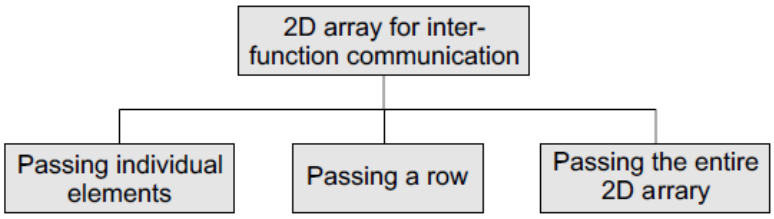


Figure 3.31 2D arrays for inter-function communication

1. Passing individual elements

- ✓ The individual elements of an array can be passed to a function by passing either their **data values or addresses.**

Passing Data Values

Calling function

```
main()
{
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    func(arr[1][1]);
}
```

Called function

```
void func(int num)
{
        printf("%d", num);
}
```

Passing Addresses

Calling function

```
main()
{
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    func(&arr[1][1]);
}
```

Called function

```
void func(int *num)
{
        printf("%d", *num);
}
```

2. Passing a Row

- ✓ A row of a two-dimensional array can be passed by **indexing the array name with the row number.**
- ✓ Look at Fig. 3.32 which illustrates how a single row of a two-dimensional array can be passed to the called function.

Calling function

```
main()
{
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    func(arr[1]);
}
```

Called function

```
void func(int arr[])
{
        int i;
        for(i=0;i<5;i++)
            printf("%d", arr[i]);
}
```

3. Passing the Entire 2D Array

- ✓ To pass a two-dimensional array to a function, we use the **array name as the actual parameter**. However, the parameter in the called function must indicate that the array has two dimensions.

Calling function

```
main()
{
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    func(arr);
}
```

Called function

```
void func(int arr[5][5])
{
        for(i=0;i<5;i++)
                for(j=0;j<5;j++)
                    printf("%d", arr[i][j]);
}
```

3.10 Multi-Dimensional Arrays

- ✓ A multi-dimensional array in simple terms is **an array of arrays.**
- ✓ As we have **one index in a one-dimensional array**, **two indices in a two-dimensional array**, in the same way, we have **n indices in an n-dimensional array or multi-dimensional array.**
- ✓ Conversely, an **n–dimensional array is specified using n indices**.
- ✓ An n-dimensional $m_1 \times m_2 \times m_3 \times \ldots \times m_n$ array is a collection of $m_1 \times m_2 \times m_3 \times \ldots \times m_n$ elements. In a multi-dimensional array, a particular element is specified by using n subscripts as $A[I_1][I_2][I_3]\ldots[I_n]$,
- ✓ Figure 3.33 shows a three-dimensional array. The array has three pages, four rows, and two columns.

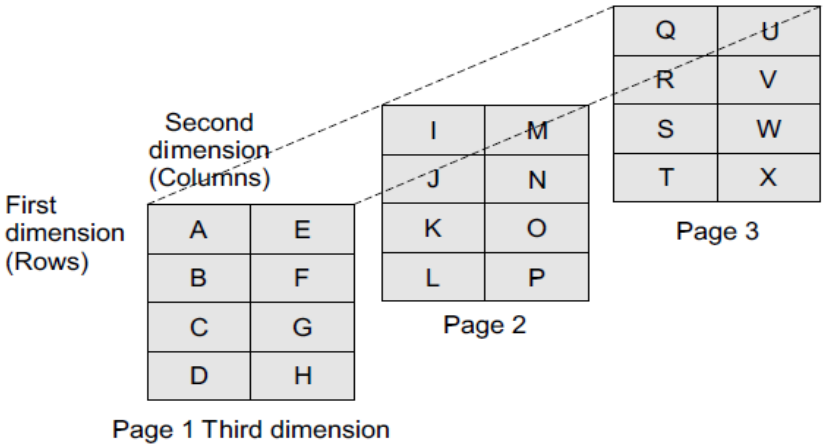


Figure 3.33 Three-dimensional array

Example:

1. Write a program to read and display a 2 × 2 × 2 array.

```
#include <stdio.h>

void main()
{
        int array[2][2][2], i, j, k;

        printf("\n Enter the elements of the matrix");
        for(i=0;i<2;i++)
        {
                for(j=0;j<2;j++)
                {
                        for(k=0;k<2;k++)
                        {
                                scanf("%d", &array[i][j][k]);
                        }
                }
        }
```

```
        printf("\n The matrix is : ");
        for(i=0;i<2;i++)
        {
                printf("\n");
                for(j=0;j<2;j++)
                {
                        printf("\n");
                        for(k=0;k<2;k++)
                                printf("\t array[%d][%d][%d] = %d", i, j, k, array[i][j][k]);
                }
        }
}
```

Output

Enter the elements of the matrix
1 2 3 4 5 6 7 8
The matrix is
arr[0][0][0] = 1 arr[0][0][1] = 2
arr[0][1][0] = 3 arr[0][1][1] = 4
arr[1][0][0] = 5 arr[1][0][1] = 6
arr[1][1][0] = 7 arr[1][1][1] = 8

3.11 Applications of Arrays

Arrays are frequently used in C, as they have a number of useful applications. These applications are

- ✓ Arrays are widely used to **implement mathematical vectors, matrices, and other kinds of rectangular tables.**
- ✓ **Many databases** include one-dimensional arrays whose **elements are records.**
- ✓ Arrays are also used to **implement other data structures such as strings, stacks, queues, heaps, and hash tables.**
- ✓ Arrays can be used for **sorting elements in ascending or descending order.**

soprotection.com